IOT PRACTICUM JOURNAL

MODULE 2

WEBSOCKET COMMUNICATION WITH A LOCAL SERVER

Group : .....

Day/Shift : .....

Instructor Assistant Code : .....

Name : .....

NIM : .....

IOT DEVICE PROGRAMMING PRACTICUM

SCHOOL OF ELECTRICAL ENGINEERING

TELKOM UNIVERSITY

2026

A. Circuit and System

1. Prepare the following components:

   - ESP32 DevKit V1 (DOIT) (1x)
   - Orange Pi 4 PRO (1x, as the local server)
   - DHT22 Sensor (1x)
   - Breadboard (1x)
   - Wires/Jumpers (as needed)

2. Build the following circuit using the tools and components you have prepared! (DHT: DHT22 Sensor)

Module 2 circuit schematic (to follow)

Client (ESP32 DevKit V1 (DOIT))		Server (Orange Pi 4 PRO)
Component	**Pin**	**Component**
DHT (VCC)	3V3	No additional circuit. The WebSocket server runs entirely in software, so the Orange Pi only needs to be on the same Wi-Fi network as the ESP32.
DHT (DATA)	GPIO16	
DHT (GND)	GND	

Note: a DHT22 must not be read more often than once every 2 seconds; a tighter interval returns `NAN`*. This module uses no actuator at all: the ESP32 only reads and sends, while the Orange Pi only records and replies with its own receive time.*

Path assignment per group. The server tells each group's board apart by its path, so several groups can send to one and the same server without disturbing each other. Use only your own group's path.

Group	`WS_PATH`	Group	`WS_PATH`
1	`/grup1`	5	`/grup5`
2	`/grup2`	6	`/grup6`
3	`/grup3`	7	`/grup7`
4	`/grup4`	8	`/grup8`

3. Before running the program, prepare your working environment in the following order. Tick every step you have completed!

   ☐ Installing the `DHT sensor library` and `Adafruit Unified Sensor` libraries through the Arduino IDE Library Manager

- ☐ Installing the `WebSockets` library by Markus Sattler through the Arduino IDE Library Manager
- ☐ Preparing the `websockets` virtual environment on the Orange Pi (see the box below)
- ☐ Noting the Orange Pi's IP address with `hostname -I`
- ☐ Matching the port and path values across both program files

Package requirement: the `websockets.asyncio.server` module only exists in `websockets` version 13 and above, while the package shipped with the system is usually older.

```
python3 -m venv ~/venv-iris
source ~/venv-iris/bin/activate
pip install "websockets>=13"
```

4. Complete the program below by filling in the blank sections. Afterwards (or while completing it), retype the blank sections you have filled in using the VS Code/Arduino IDE application (for the ESP32) and the text editor on the Orange Pi (for the Python server) prepared by your respective lab assistants!

   Blank sections are marked with an underscore run `________` and numbered `(1)` through `(6)` in the comment to their right. Replace each underscore run with the correct code following the hint in its comment. Note that the **port** and **path** values on the Python server must match the values used in the ESP32 code exactly.

Server Source Code — WebSocket (Python, Orange Pi)

```
from websockets.asyncio.server import ServerConnection, serve
from websockets.exceptions import ConnectionClosed

HOST: str = "0.0.0.0"  # Listens on every network interface
PORT: int = ________  # (1) Port number, same as WS_PORT in every ESP32 sketch

# Accepted paths, one per group. Eight are provided; delete the unused ones or
# add new ones if the number of groups changes.
ALLOWED_PATHS: set[str] = ________  # (2) Accepted paths, same as each ESP32's WS_PATH


def timestamp() -> str:
    """Returns the current wall clock in HH:MM:SS:MS format."""
…
            received_at = timestamp()

            # The message arrives as JSON text and must be turned into a dict
            # before its values can be read.
            reading = ________(message)  # (3) Function that parses JSON text into a dict

            seq = int(reading["seq"])
            temperature = float(reading["temp"])
            humidity = float(reading[________])  # (4) Humidity JSON key, same as on the
ESP32

            print(f"[{received_at}] {group} #{seq}: {temperature:.1f} C, "
                  f"{humidity:.1f} %")
…
            # The reply is an acknowledgement only: the sequence number and the
            # server-side receive time, so the two can be compared in the journal table.
            response = json.dumps({"seq": seq, "at": received_at})
            await websocket.________(response)  # (5) Method that sends the reply

    except ConnectionClosed:
        print(f"[{timestamp()}] {group} disconnected")
…
async def main() -> None:
    # ping_interval/ping_timeout close a connection that no longer answers a
    # pong, so a "zombie" connection cannot leave the ESP32 waiting forever.
    runner = ________  # (6) Function that runs the server
    async with runner(
        handle_client, HOST, PORT, ping_interval=15, ping_timeout=10
```

```
    ):
        print(f"[{timestamp()}] WebSocket server running on {HOST}:{PORT}")
```

Client Source Code — WebSocket (ESP32)

```
 * beside each one are the hints.
 */

#include <WiFi.h>
#include <________>  // (1) WebSocket client library header (arduinoWebSockets)
#include <DHT.h>
#include <time.h>
#include <sys/time.h>  // For microsecond-resolution gettimeofday()
…
// This path is what tells the server which group's board is connecting.
// Use only your own group's path: "/grup1", "/grup2", "/grup3", "/grup4",
// "/grup5", "/grup6", "/grup7", or "/grup8". The value must be present in
// ALLOWED_PATHS on the server.
const char*    WS_PATH = "________";  // (2) Path, same as on the Python server

// NTP: the wall-clock synchroniser. An offset of 25200 seconds equals WIB (UTC+7).
const char*  NTP_SERVER     = "pool.ntp.org";
const long   GMT_OFFSET_SEC = 25200;
…
// Builds one JSON frame holding the latest DHT22 reading. Returns an empty
// string when the sensor cannot be read.
String buildReading() {
  float humidity    = dht.readHumidity();
  float temperature = dht.________();  // (3) Method that reads temperature in Celsius

  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("DHT22 could not be read, check its data wire and power supply.");
    return "";
…
  connectToWiFi();
  syncClock();

  webSocket.begin(WS_HOST, WS_PORT, WS_PATH);
  webSocket.onEvent(________);  // (4) Name of the callback above, without parentheses

  // Ping every 15 seconds; after 2 pings go unanswered within 3 seconds the
  // board drops the stalled connection itself and auto-reconnect takes over.
  webSocket.enableHeartbeat(15000, 3000, 2);
…
  Serial.println("DHT22 readings will be sent every 2 seconds.");
}

void loop() {
  webSocket.________();  // (5) Event-processing method, must be called regularly

  if (!webSocketConnected) {
    return;
  }
…
  sendTime = millis();
  waitingForReply = true;

  webSocket.________(message);  // (6) Method that sends a text message

  Serial.printf("[%s] Send #%lu: %s\n",
                wallClockTimestamp().c_str(), seq, message.c_str());
}
```

5. Record the configuration you used in the following table! The top three values must match exactly between the Python server file and the ESP32 file, otherwise the WebSocket connection will never form.

Parameter	Value used
Orange Pi IP address (`WS_HOST` / `hostname -I`)	
Server port (`PORT` / `WS_PORT`)	

Parameter	Value used
WebSocket path (`WS_PATH`)	
Wi-Fi network name (`WIFI_SSID`)	
DHT22 data pin on the ESP32 (`DHT_PIN`)	
Sampling interval (`SAMPLE_INTERVAL_MS`)	

B. Analysis

1. Draw the communication flow implemented in the system you have worked on!

2. Next, a response-time experiment will be carried out for data sent from the **Client (ESP32)** to the **Server (Orange Pi)**. First open the Serial Monitor on the ESP32 and the Python server terminal on the Orange Pi. Every 2 seconds the ESP32 prints its **send time** with a sequence number and the DHT22 reading, while the Orange Pi terminal prints the **receive time** with the same sequence number, temperature and humidity; that number is what pairs a line on one display with a line on the other. The server's reply carries no command, only the sequence number and its receive time in the `at` field, so both times can still be read from the ESP32 Serial Monitor alone. Record:
   - **FIVE (5)** pairs of readings: the send time, temperature and humidity from the ESP32 Serial Monitor, together with the receive time from the Orange Pi terminal

   Both times are recorded in **HH:MM:SS:MS** format. Compute the **one-way delay** by subtracting the send time from the receive time, then compare it against the round-trip delay the ESP32 prints for itself.

   Note: the ESP32 and the Orange Pi are on the same Wi-Fi network, so both clocks are synchronised over NTP (`configTime()` on the ESP32, systemd-timesyncd on the Orange Pi). That means times from the two displays really can be subtracted and the result is a genuine one-way delay. NTP accuracy over Wi-Fi is around 10–50 ms, so it is normal for your one-way delay not to come out at exactly half the round-trip delay.

Trial no. …	Temp. [°C]	Humidity [%]	Send time *(ESP32)* **[HH:MM:SS:MS]**	Receive time *(Orange Pi)* **[HH:MM:SS:MS]**	One-way delay **[ms]**	Round-trip delay **[ms]**
1.						

Trial no. …	Temp. **[°C]**	Humidity **[%]**	Send time *(ESP32)* **[HH:MM:SS:MS]**	Receive time *(Orange Pi)* **[HH:MM:SS:MS]**	One-way delay **[ms]**	Round-trip delay **[ms]**
2.						
3.						
4.						
5.						
Average delay (ms)						

3. Write your **analysis** of the system you experimented with earlier in the fill-in section below!

(Also give the **relationship/correlation** by applying the **communication methodology** you are studying in this module; it does not have to be complete!)

C. Suggestions and Feedback

1. Assistant

2. Laboratory